```
  funcA();
  funcB();
  funcC();
}
```

Assume that the code modification was to remove the *lock* and *unlock* calls in '*funcA*' and '*funcC*'. Now, if any thread is executing '*funcB*', as per our safety criterion, none of the threads have the 'active functions' on their stack, and hence the system can be patched. Having patched the application, when the thread next enters '*funcC*', it goes to the new version—which does not contain the *unlock* call. But, this particular thread earlier executed the *older* version of '*funcA*', and has locked *L1*. Since there is no *unlock* call in the new '*funcC*', the lock remains held by the thread—which is an error.

Therefore, our simple safety criterion is not sufficient to guarantee the safety of the DSU. We will continue our discussion of DSUs in next month's column, focusing on their safety and timeliness.

This month's takeaway question

Our takeaway problem is also related to the DSU problem. Let us consider the issue of modifications to data structures whose instances have already been created. For instance, consider a linked list of objects, which is created at application start-up, and accessed throughout the application lifetime. As part of a defect fix, the developer added a new field to each object, and the DSU contains code that sets/gets the added field. However, this newly added field is not a standalone variable; it is part of an existing data structure, which has already been allocated by the application, and references to which are held by other objects. If necessary, can we dynamically update the application with the modified type for this object? What are the possible techniques?

This question is one to ponder over, rather than a programming problem. Can you think of scenarios in which an application cannot be updated dynamically? Can you write a tool that can verify whether a dynamically updated application is indeed performing correctly?

My 'must-read book' for this month

This month's must-read book is 'Algorithms for Interviews' by Aziz and Prakash. This is an entertaining book, along the lines of Steve Skiena's 'Algorithm Design Manual', discussing difficult programming problems and their algorithmic solutions. However, be aware that this is not meant to be an introduction to algorithms. As the book title suggests, this is a very useful book to browse through if you are preparing for interviews at Google, Microsoft or Amazon.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in the column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming, and here's wishing you the very best. END

By: Sandya Mannarswamy

The author is an expert in systems software and works at Hewlett-Packard India. Her interests include compilers, virtualisation technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's website http://www.tech-pathshala.com, which offers practice interviews and training for jobs in this field, and her LinkedIn group, tech-pathshala' s systems software training group at *http://www.linkedin.com/groups?home=&gid=3813966&trk=anet_ug_hm*.